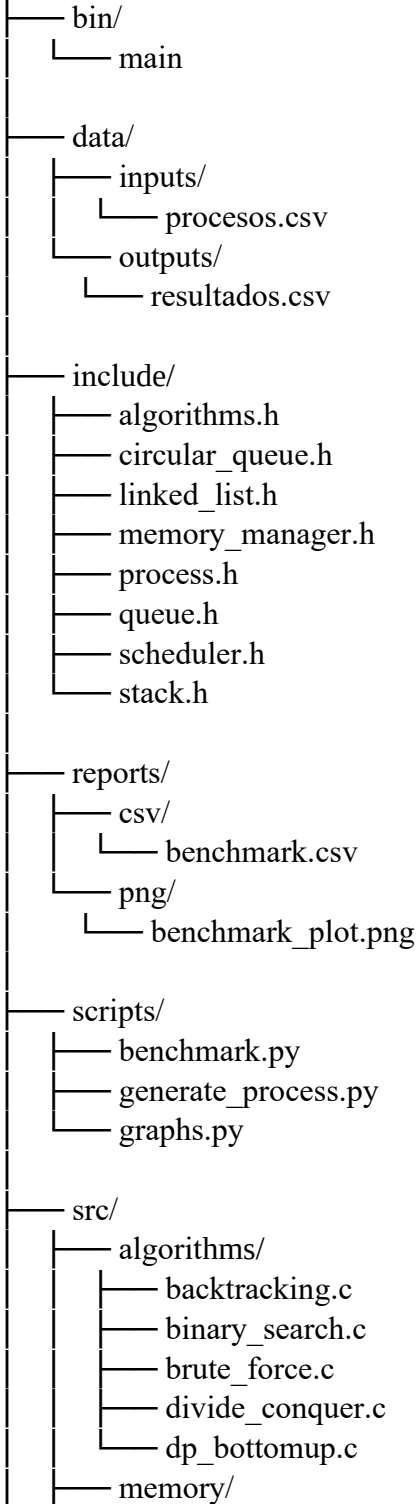
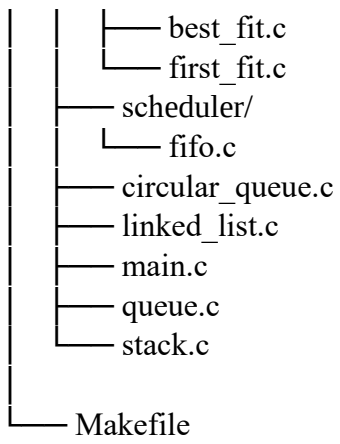


Simplificacion de Reporte Tecnico

Orden de los programas

proyecto-final/





Orden con que hace cada parte.

proyecto-final/

|
|— **bin/**
| |— **main** (El programa final de unos y ceros listo para que la computadora lo corra).

|
|— **data/**
| |— **inputs/**
| |— **procesos.csv** (El archivo de texto donde escribes a mano los procesos que quieres probar).

| |— **outputs/**
| |— **resultados.csv** (Aquí el C escupe su reporte después de correr la simulación).

|— **include/** (Los "contratos" o índices que conectan las piezas de código entre sí).

| |— **algorithms.h**
| |— **circular_queue.h**
| |— **linked_list.h**
| |— **memory_manager.h**
| |— **process.h**
| |— **queue.h**
| |— **scheduler.h**
| |— **stack.h**

|— **reports/**
| |— **csv/**
| |— **benchmark.csv** (El archivo que crea Python midiendo cuántos milisegundos tardó tu código).

| |— **png/**
| |— **benchmark_plot.png** (La imagen de la gráfica azul generada por Python).

|— **scripts/**
| |— **benchmark.py** (Script que pone a sudar al programa en C mandándole miles de procesos).

| |— **generate_process.py** (Script que inventa procesos aleatorios rápido para no escribirlos a mano).

| |— **graphs.py** (Script que dibuja la gráfica visual leyendo el benchmark.csv).

|— **src/**
| |— **algorithms/**
| |— **backtracking.c** (Guarda intentos de memoria en una pila y retrocede si el intento falla).

```

|  |  | — binary_search.c  (Busca un proceso rapidísimo partiendo la lista a la mitad
|  |  | — brute_force.c    (Busca el peor hueco de memoria revisando todo de principio
|  |  | — divide_conquer.c (Ordena la memoria: avienta los ocupados a un lado y los
|  |  | — dp_bottomup.c     (Calcula inteligentemente qué procesos son más valiosos
|  |  | — memory/
|  |  | — best_fit.c       (Busca el hueco de RAM más apretado para no dejar casi
|  |  | — first_fit.c      (Toma el primer hueco de RAM que le sirva y no pierde tiempo
|  |  | — scheduler/
|  |  | — fifo.c           (Aquí viven las reglas para ordenar procesos por FIFO, Round
|  |  | — circular_queue.c (Una fila en forma de círculo infinito para que el Round
|  |  | — linked_list.c    (Una lista de "vagones de tren" para guardar los procesos que
|  |  | — main.c           (EL CEREBRO: Prende el reloj, junta todas las piezas y corre el
|  |  | — queue.c          (Una fila normal donde el primero que llega es el primero en
|  |  | — stack.c          (Una pila tipo platos para ayudar al algoritmo de Backtracking).
|  |  | — Makefile         (El manual de instrucciones para que la PC compile todo con solo
|  |  |                      escribir "make").

```

Resumen de Complejidad Algoritmica

- **Búsqueda Binaria** - Divide y Vencerás - Teorema Maestro - **$O(\log n)$**
- **Compactación de Memoria** - Divide y Vencerás - Sumatoria - **$O(n)$**
- **Worst Fit (Peor Ajuste)** - Fuerza Bruta - Sumatoria - **$O(n)$**
- **Asignación de Memoria con Pila** - Backtracking - Sumatoria - **$O(n)$**
- **First Fit (Primer Ajuste)** - Greedy (Avaro) - Sumatoria (Peor Caso) - **$O(n)$**
- **Best Fit (Mejor Ajuste)** - Greedy (Avaro) - Sumatoria - **$O(n)$**
- **Shortest Job First (SJF)** - Greedy (Avaro) - Sumatoria - **$O(n)$**
- **Mochila (Knapsack)** - Programación Dinámica (Bottom-Up) - Doble Sumatoria - **$O(n * W)$**
- **Planificador FIFO** - Estructura Base (Cola Lineal) - Conteo de Instrucciones - **$O(1)$**
- **Planificador Round Robin** - Estructura Base (Cola Circular) - Conteo de Instrucciones - **$O(1)$**

Conclusion general del sistema

La complejidad asintótica general del sistema está dominada por su algoritmo más pesado, la Programación Dinámica, dándole un límite superior teórico de **$O(n \times W)$** . Sin embargo, la arquitectura está diseñada para que este esfuerzo se realice solo durante la optimización inicial. Durante el ciclo de vida continuo del procesador, gracias a las Colas Circulares $O(1)$ y la Búsqueda Binaria $O(\log n)$, el cuello de botella en tiempo real se reduce exclusivamente a la búsqueda de memoria **$O(m)$** , logrando un simulador altamente escalable y eficiente.

Benchmark

```
import subprocess
import time
import csv

def run_benchmark():
    # Diferentes cargas de procesos para medir escalabilidad
    tamanos = [100, 500, 1000, 5000]
    resultados = []

    print("Iniciando Benchmark...")
    for n in tamanos:
        # Medimos el tiempo que tarda el ejecutable de C
        inicio = time.time()
        # Llamamos al ejecutable (asegúrate de que esté en bin/main)
        subprocess.run(["./bin/main.exe", str(n)], stdout=subprocess.DEVNULL)
        fin = time.time()

        tiempo_ms = (fin - inicio) * 1000
        resultados.append([n, tiempo_ms])
        print(f"Carga de {n} procesos completada en {tiempo_ms:.2f} ms")

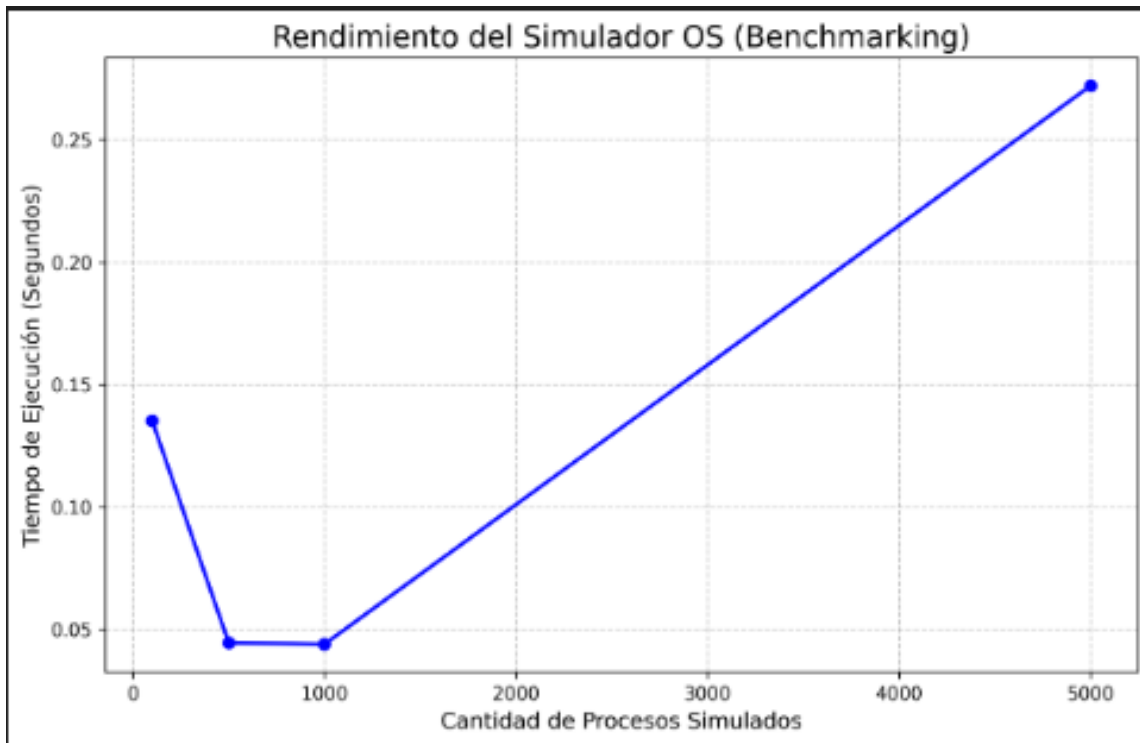
    # Guardamos los resultados
    with open("reports/csv/benchmark.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Procesos", "Tiempo_ms"])
        writer.writerows(resultados)
    print("Benchmark guardado en reports/csv/benchmark.csv")

if __name__ == "__main__":
    run_benchmark()
```

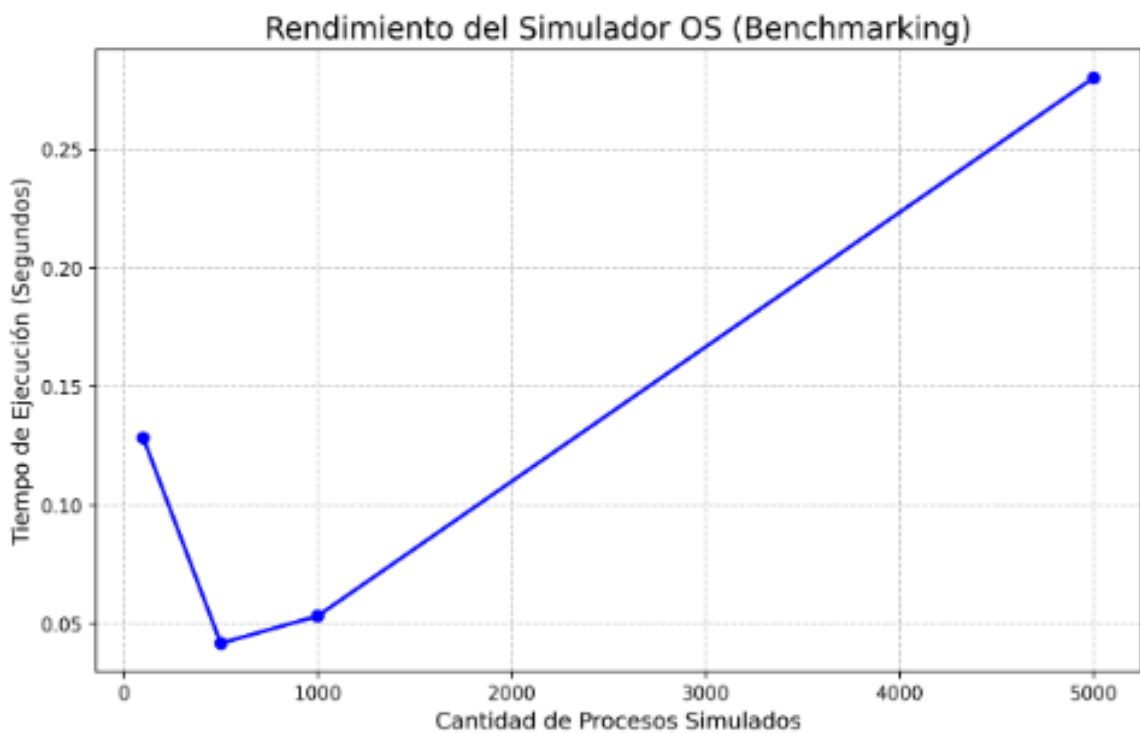
Graficas

Bajo las mismas condiciones de recursos, siendo una RAM de 1024 y Quantum de 2
Simulando 100, 500, 1000 y 5000 procesos

Con el first fit



Con el best fit



El first fit corresponde mejor a mayor cantidad de datos

Conclusion

Para esta prueba sencilla, se observan la diferencia entre los algoritmos en concreto, su velocidad a la hora de manejar datos tan grandes, aun que si son mas pequenos estos son básicamente iguales con la unica diferencia que el First Fit tiene una velocidad de “arranque” , como si fuera una computadora recién encendida, por ello es un poco menos veloz con datos pequenos

- **First Fit** es ligeramente más rápido (0.272s) porque simplemente recorre la memoria y en el momento que encuentra un hueco donde el proceso cabe, lo mete ahí y deja de buscar. Pierde menos tiempo procesando.
- **Best Fit** es un poco más lento (0.280s) porque, aunque encuentre un hueco rápido, tiene la obligación matemática de seguir buscando por **toda** la memoria para asegurarse de encontrar el hueco "más perfecto" (el que deje menos espacio desperdiciado). Esa búsqueda extra le cuesta tiempo de CPU.